Basic (your first LCEL chain) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `lcel_chain_basics_practice.py`: this 3-piece chain — a `ChatPromptTemplate`, a chat model, and an output parser, joined with `|` — is the atom every LangChain thing you'll ever build is made of. The `|` works like a Unix pipe: each step's output becomes the next step's input, so the prompt turns your variables into messages, the model turns those messages into a reply, and the parser turns that reply into a plain string (and, later, into a structured object). Writing it here once, on its own, is what makes the `|` syntax and each piece's job obvious before Doc06's tools, Doc08's RAG pipeline, or this document's own Build Task pile more onto it. **If not:** the first time you'd see `prompt | model | parser` would be inside a bigger chain already doing three other things, making it hard to tell which part the `|` syntax actually changed.

Solution

```
# lcel_chain_basics_practice.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

# The atom of every LangChain app: three pieces, joined with |.
prompt = ChatPromptTemplate.from_template("Answer: {question}")
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
parser = StrOutputParser()

# | feeds the prompt's output into the model, and the model's into the parser
chain = prompt | model | parser

result = chain.invoke({"question": "What's 5 + 7?"})
print(f"Answer: {result}")
```

Expected output:

```
Answer: 5 + 7 = 12
```